

操作系统第七周星期一讲稿整理

 **考试提醒：**4月29日期中考试，范围考到进程部分（死锁之前可能不考）

 **实验提醒：**Lab3进程管理、Lab4 fork实现

课前说明

关于Live2实验

- 去年Live2的exam题目做成了今年的预习题，**建议大家做一遍**
- Live2最核心、最有挑战性的是**内存管理**部分
- 特别是用**红黑树**管理页表的代码，一定要仔细看
- 这部分代码是早期Linux高手写的，设计非常精巧
- 重点体会：通过指针的指针如何操纵双向链表

链表操作实践建议

数据结构课学过链表，但操作系统里的链表实现非常精巧，建议大家：

1. 把指导书里的链表操作代码做熟练
2. 自己实现：遍历、循环、头插、尾插、中间插
3. 练习检索存在/不存在的元素

C语言与Rust的对比

为什么Linux用C写而不是Rust？

- C语言可以非常贴近硬件，指针操作非常灵活
- Rust要保证内存安全，语法结构严谨但灵活性受限
- **指向指针的指针**这种技巧在Rust里不容易实现

 **学习建议：**学完硬件内存和地址概念后，反过来理解C语言指针，会有更深的认识。

一、进程控制的基本概念

1.1 进程的组织方式

树状结构（UNIX/Linux采用）

在UNIX类操作系统中，所有进程组织成一棵**进程树**：

- 0号进程是最根的进程
- 由0号进程创建1号进程
- 剩下的所有进程都是1号进程的子孙

树状结构的优势：

- 用单一根节点就可以管理整个系统的所有进程
- 天然表达了进程创建的父子关系
- 便于实现进程的资源继承和权限管理

链表结构

用链表也可以管理所有进程，维护一个链头就行。

树状 vs 链表对比：

特性	树状结构	链表结构
语义表达	能表达父子创建关系	仅线性排列，无额外语义
管理复杂度	稍复杂	简单
扩展性	支持分叉，更灵活	一维线性

 Windows不是用fork创建进程的，而是用CreateProcess，所以Windows的进程组织方式不同。

1.2 原语的概念

原语 (Primitive)：由若干条指令组成，用于完成特定功能的原子操作。

原子性的核心含义：

- 要么全部执行完成
- 要么完全不执行
- **不能执行到一半被打断**
- 一旦被分割就会破坏语义完整性

为什么进程控制需要原语？

- 创建进程不能创建一半
- kill进程不能杀得半死不活
- 状态切换不能中途暂停，否则会出现不一致

原语 vs 系统调用

关系	说明
交集	进程控制相关的基本操作几乎都是原语（如fork、exit等）
区别	不是所有系统调用都是原语，有些系统调用可以中途暂停后继续执行

1.3 fork()的实现机制

fork()作为一个C函数，它的实际调用链：

用户态C代码调用 `fork()`
↓（函数实现在libc标准库）

封装Syscall指令 / int中断

↓ (陷入内核)

OS内核处理fork逻辑

为什么要在标准库里封装？

- 操作系统是运行时状态，上层应用可能用各种语言编写（C、汇编、Python等）
- 操作系统统一暴露Syscall指令或int中断作为接口
- 陷入内核的过程比较复杂，涉及特权级切换
- 不想把这些硬件细节暴露给所有用户
- 所以C语言库把这些细节封装成友好的函数接口

💡 大家做Lab4的时候会对这个理解更深。

二、fork()原语详解 ★

2.1 fork()的神奇语义

示例代码：

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork(); // ← 调用点，从这里"分叉"

    if (pid < 0) {
        printf("fork出错了! \n");
    } else if (pid == 0) {
        printf("我是儿子进程\n"); // 子进程走这个分支
    } else {
        printf("我是父进程\n"); // 父进程走这个分支
    }

    printf("这句话会打印两次\n"); // 两个进程都会执行到这里
    return 0;
}
```

运行结果：

- 两个分支都会打印！
- 最后那句话会打印两次！

⚡ 关键理解：一个函数被执行完后，出现了两个返回值 —— 但实际上是在两个不同的进程地址空间里，各有一个返回值。

2.2 fork()执行前后的变化

在fork()调用点之前：

- 系统中只有一个进程在执行这段代码

在fork()调用点之后：

- 变成了两个进程
- 两个进程都从fork()的返回点继续往下执行
- 它们是**并发执行**的关系

2.3 父子进程的异同点

相同点（子进程继承自父进程）	不同点
整个内存空间的内容	PC程序指针不同（走向不同分支）
所有通用寄存器的值（除了返回值寄存器）	进程ID不同
打开的文件描述符	fork()的返回值不同
程序的执行上下文	父进程返回子进程PID，子进程返回0

 实践建议：把这段fork代码贴到C编辑器里编译运行，实际感受一下并发创建进程的效果。

2.4 fork()的实现思路（Lab4要做）

从操作系统内核视角看，实现fork()大概分几步：

1. **分配新的PCB**：为子进程创建一个新的进程控制块
2. **复制上下文**：把父进程的PCB内容大部分复制给子进程
3. **特殊设置**：
 - 子进程的PC指针设置成fork返回点
 - 设置子进程的返回值寄存器为0
 - 给子进程分配新的唯一PID
4. **加入就绪队列**：把子进程的PCB插到就绪队列里，等待调度
5. **返回用户态**：父进程正常返回，返回值是子进程PID

 对于OS来说：进程 = PCB数据结构。有了PCB，OS就可以管理和调度这个进程了。

三、进程三态模型

 **敲黑板重点**：三态模型每次考试都考，状态转换图一定要掌握！

3.1 三态的定义

状态	英文	核心含义	描述
就绪状态	Ready	万事俱备，只欠CPU	除了CPU之外什么都不缺，所有运行条件都满足，只要调度到就能立刻跑
执行状态	Running	正在CPU上跑	占用CPU资源正在执行指令

状态	英文	核心含义	描述
阻塞状态	Blocked / Waiting	等待CPU以外的条件	不具备执行条件，比如在等用户输入、等I/O完成、等信号量

为什么需要定义这三态？

因为多道程序并发有个硬约束：

一次只能有等于CPU个数的进程在执行

其余具备执行条件的进程需要排队等CPU → 需要**就绪队列**来管理

不具备执行条件的进程不能占着就绪队列 → 需要**阻塞队列**来另外管理

3.2 状态转换图（必考！）



四条转换边详解：

转换方向	触发条件	具体场景说明
就绪 → 执行	调度 (Scheduler)	调度器从就绪队列里挑一个进程，分配CPU给它
执行 → 就绪	时间片到	进程用完了分配给它的时间片，回到就绪队列继续排队
执行 → 阻塞	等待某事件	执行中发现需要等外部条件：等用户按键、等磁盘I/O、等另一个进程发信号
阻塞 → 就绪	等待的事件发生了	等待的条件满足了：用户按了键、I/O完成了、信号发来了

⚠ 两条绝对不存在的边（特别容易考到）：

1. ❌ 就绪 → 阻塞：不可能！

- 原因：就绪状态的进程根本没在执行，不会发现需要等新条件
- 进程只有在执行过程中才能往前推进，才会发现"我需要等什么"

2. ❌ 阻塞 → 执行：不可能！

- 原因：阻塞唤醒后必须先到就绪队列排队，经调度才能执行
- 要公平：其他进程也在就绪队列等了很久，不能让刚唤醒的直接插队

3.3 三态模型的扩展

更完整的商业操作系统可能在三态基础上增加更多状态：

扩展状态	作用
创建状态	新进程正在初始化，还没加入就绪队列
挂起就绪	进程被换出到外存了，即使被调度也跑不了
挂起阻塞	外存上的阻塞进程
终止状态	进程已经退出，但还没被父进程回收资源（僵尸态）

 **课程要求：**重点掌握核心三态及其四条转换关系，扩展状态了解即可。

四、进程控制块（PCB）★

4.1 PCB的核心地位

PCB（Process Control Block）是操作系统维护进程信息的**最基本、最重要的数据结构**。

对于操作系统内核来说：

进程操作	对应PCB操作
创建一个进程	创建并初始化一个PCB
删除一个进程	回收、释放PCB
切换进程状态	修改PCB里的状态字段
调度选进程	遍历就绪队列里的PCB，选一个最合适的
唤醒阻塞进程	把PCB从阻塞队列移到就绪队列

 **核心观点：**对于OS来说，进程 = PCB数据结构

管理进程 = 操作PCB + 配套的上下文切换机制

4.2 PCB必须包含的信息

一个PCB至少要包含：

类别	具体内容
标识信息	进程ID（PID）、父进程ID、用户ID（UID）、组ID（GID）
CPU状态	所有通用寄存器的值、PC程序计数器、程序状态字PSW、栈指针
调度信息	当前状态（就绪/执行/阻塞）、调度优先级、等待时间统计
内存信息	页表指针、地址空间分布、内存使用统计
I/O信息	打开的文件描述符表、IO设备使用情况
统计信息	累计执行时间、资源使用记账

4.3 PCB名字的由来

为什么叫"块" (Block) ?

- 这是历史遗留的叫法
- 早期硬件能力差，内存都是按块管理的
- 一个PCB刚好fit到一个内存块上

这个"块"的叫法就一直延续下来了，比如内存也有内存控制块。

五、实验关联

Lab3（进程管理）要实现的内容：

1. 进程创建原语
2. 进程删除原语
3. PCB数据结构的管理
4. 进程状态切换机制
5. 调度算法的实现

Lab4要实现的内容：

- fork()原语的完整实现

本章考点总结

考点	考察形式	重要程度
fork原语的语义	选择题、简答题	☆☆☆
父子进程的异同点	选择题、简答题	☆☆
进程三态的定义	名词解释、选择题	☆☆
三态转换图（四条边、两条不存在的边）	画图题、选择题	☆☆☆
PCB的作用和必须包含的内容	简答题	☆☆
原语的原子性含义	名词解释	☆☆